JoySeminar

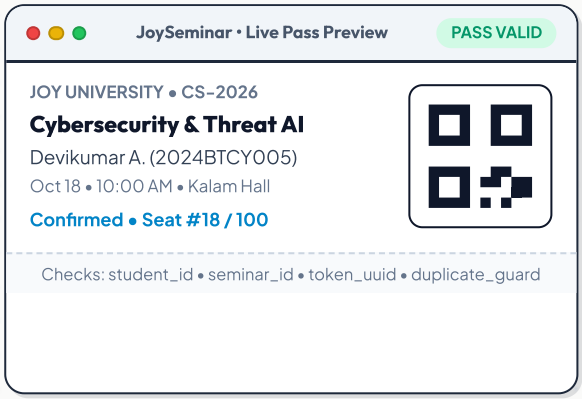
Intelligent University Seminar Registration & Verification Platform

Project Report | Joy University | Guide: Ambika Ma'am | Academic Year: 2026-2027 | Devikumar Anegondhi [2024BTCY005] | Department of CSE (Cybersecurity)

1 What is JoySeminar?

A centralized campus web workspace to organize, discover, reserve, and verify university seminars — attendee quotas, digital QR entry passes, instant device-camera attendance, waitlist auto-promotion, and tamper-proof PDF certificates consolidated in one place.

- Seminar Discovery
- Encrypted QR Pass
- Live QR Scanner
- Instant Check-in
- Auto-Certificates
- Public Verification
- FIFO Waitlist
- Role-Based Access
- Live Analytics
- Automated Alerts



2 Problem > Solution

THE TRADITIONAL PROBLEM

- Paper passes easily lost, forgotten, or shared.
- Slow manual paper sign-in queues causing congestion.
- Fake, forged, or unverified participation certificates.
- Overbooked seminar halls beyond fire code safety limits.
- No automated waitlists when registered students don't show.

→

JOYSEMINAR FIXES

- Single-use cryptographic QR passes rendered on mobile.
- Sub-second device camera QR scanner at venue doors.
- Tamper-proof PDF certificates with live public hash verify.
- Strict concurrency locking preventing overbooking.
- Automated FIFO waitlist promotion upon cancellations.

3 Main Goal

> Build a centralized university seminar platform where academic events are organized, seats are reserved with digital QR passes, attendance is verified instantaneously via camera scanning, and tamper-proof certificates are generated and validated.

4 System at a Glance

University Users	Browser UI	Client State	Express REST API	PostgreSQL DB
Admin • Speaker • Student	React 19 • Tailwind • Vite	AuthContext • QR Scanner	/api/seminars • /attendance	Normalized Relational Data

PDFKit & QR Engine
On-the-fly vector pass & certificate synthesis with embedded 2D QR matrix tokens.

Engineering Challenge: Enforce strict auditorium capacity, eliminate duplicate check-ins, automate waitlist promotions, and issue cryptographically verifiable certificates in sub-second latency.

1 Functional Requirements

1. Seminar Discovery
Browse upcoming talks, speaker bios, topics, venues, and live seat counters.

2. Seat Reservation
One-click student booking with transactional capacity enforcement.

3. Digital QR Pass
Unique cryptographic QR token generated and downloadable as A5 entry pass.

4. Live Camera Scanner
Browser-based HTML5 camera scanner for organizers at seminar hall doors.

5. Duplicate Check Guard
Rejects re-scanned passes instantly with exact original check-in timestamp.

6. FIFO Waitlist Engine
Automated queue promotion and alert when a registered student cancels.

7. Automated Certificate
Instant PDFKit vector certificate generation upon verified check-in.

8. Verification Portal
Public authenticity route (/verify-certificate/:id) for recruiters & employers.

9. Role Access Control
Granular permissions: Student, Event Organizer Admin, and Super Admin.

10. Student Passbook
Student dashboard displaying active passes, past attendance, and earned certificates.

11. Analytics & Reports
Visual breakdown of attendance rate, department share, and hall capacity utilization.

12. Notification Pipeline
In-app toast notifications and transactional email confirmations via Nodemailer.

2 Non-Functional Requirements

Performance
Sub-100ms camera scan verification & dynamic PDF streaming.

Security
Bcrypt password hashing (10 salt rounds), JWT auth, and Helmet HTTP protection.

Reliability
ACID transactions with row locks (FOR UPDATE) to prevent overbooking.

Usability
Responsive Tailwind UI with clear audio/visual feedback during camera scans.

Scalability
Indexed foreign keys and connection pooling for concurrent seat bursts.

Maintainability
Modular Express MVC controllers, reusable React TypeScript components.

Compatibility
Desktop & mobile web browsers (Chrome, Edge, Safari, Firefox).

Data Integrity
Unique constraints on QR tokens, certificate IDs, and student attendance.

3 Who uses it? (Role categories)

S — Super Admin
Global settings, user permissions, audit logs & system-wide analytics.

E — Event Admin
Seminar publishing, quota configuration, live camera QR attendance scanner.

A — Student Attendee
Seminar browsing, 1-click booking, pass retrieval, certificate downloads.

V — External Verifier
Public portal query by Certificate ID or QR scan to confirm credential validity.

Role enforcement is verified on every API request via JWT payload decoding and strict Express authorization middleware.

4 Key Use Cases

1. Publish Seminar
Admin configures speaker, venue, registration deadlines, and seat limit.

2. Instant Booking
Student books seat; receives cryptographically signed digital pass.

3. Camera Check-In
Organizer scans pass at entrance; attendance marked 'present' in DB.

4. Verify Certificate
Attendee downloads vector PDF; recruiter verifies hash on public portal.

1 Architecture

University Users
Students • Event Admins • Super Admins • Public Verifiers

▼

Browser Interface
React 19 • TypeScript • Vite • Tailwind CSS • Lucide Icons

▼

Client State Management
AuthContext • HTML5-QRCode Scanner • Axios Interceptors

▼

Express REST API
Node.js / Express • JWT Guards

PDFKit & Services
Vector Certs • QR Engine

▼

PostgreSQL Database
`seminar_system_db` • Connection Pool (`pg`) • Row Locks

2 Key API Endpoints

`/api/seminars`
Catalog, topics, capacity quotas, admin CRUD

`/api/attendance`
Camera QR scanning, live check-in logs

3 Data Flow

1 Student submits booking on React frontend.
2 Express verifies JWT; checks seat availability with lock.
3 PostgreSQL commits confirmed registration & QR token.
4 Organizer scans student pass via HTML5 camera at door.
5 Backend checks duplicate guard; marks attendance.
6 PDFKit streams certificate; public verify record activates.

4 Complete Workflow

1. **User Authentication**
   Role-based login issuing secure JWT tokens.
2. **Seminar Publishing**
   Admin defines venue, speaker, and strict capacity.
3. **Discovery & Filtering**
   Students search by topic, department, and date.
4. **Instant Seat Booking**
   Transactional capacity check and pass issuance.
5. **Automated Waitlist Queue**
   If full, students queued in strict FIFO order.
6. **Camera QR Scanning**
   Door attendants scan mobile passes with cameras.
7. **Duplicate Conflict Guard**
   Prevents pass-sharing by locking to single check-in.
8. **Vector Certificate Synthesis**
   PDFKit builds tamper-proof landscape certificates.
9. **Public Credential Verification**
   Employers verify credentials via public QR link.

5 Sample Setup - Joy University

Joy University Campus | Auditorium & Joveena Block | Hall Capacity: 50 - 250 Seats | B.Tech CSE & Cybersecurity

Courses: 24BTCY151-156 | Public Route: /verify-certificate/:id

Implementation Note: Joy University deployment runs with strict JWT expiration (7 days), connection pooling with PostgreSQL 14+, and asynchronous transactional email dispatch via Nodemailer.

1 12 Core Modules

1. Seminar Discovery
Search, filter by topic & speaker profiles

2. Seat Booking Engine
Atomic quota allocation & pass creation

3. QR Pass Generator
Dynamic matrix passes with UUID tokens

4. Live Camera Scanner
HTML5 camera scanner for door staff

5. Attendance Engine
Check-in logs & duplicate collision guard

6. Certificate Synthesizer
Vector PDF generation with university seals

7. Verification Portal
Public credential check via certificate hash

8. FIFO Waitlist Pipeline
Auto-promotes students when seats open up

9. Student Dashboard
Passbook for active tickets & certificates

10. Admin Event Console
Full lifecycle management & capacity editing

11. Analytics & Reports
Department turnout & check-in charts

12. Alert Dispatcher
In-app badges & transactional Nodemailer emails

2 Data & Storage Architecture

PostgreSQL Relational Storage

- Normalized 3NF relational schema in `seminar_system_db`.
- Transactional row-level locking (`SELECT ... FOR UPDATE`).
- Foreign key cascades on registrations, attendance & waitlist.
- Indexed lookups on `qr_code_token`, `certificate_id` & emails.

Stateless JWT & Client Caching

- Cryptographically signed Bearer JWTs with 7-day expiration.
- React Context state for live user session & notifications.
- Client-side localStorage pass caching for offline viewing.
- Axios HTTP interceptors with auto-token injection.

3 7 Core Database Entities

`users` `seminars` `registrations` `attendance` `certificates` `waitlist` `notifications`

4 How they Connect (Entity Relationships)

```
users ──[1:N]──> registrations <──[N:1]── seminars | registrations ──[1:1]──> attendance | registrations
                                                  ──[1:1]──> certificates
```

Relation Logic: `Registration = User_ID * Seminar_ID * Unique_QR_Token * Status`

5 Implementation & Deployment

App.tsx · StudentDashboard.tsx · AdminAttendance.tsx · VerifyCertificate.tsx · server.js · schema.sql · pdfService.js · qrService.js · waitlistService.js

Tech Stack: Frontend: React 19 + TypeScript + Vite + Tailwind CSS. Backend: Node.js + Express REST API. Database: PostgreSQL 14+. Vector PDF & QR: PDFKit & node-qrcode.

1 QR Verification & Attendance Pipeline

```
PROCEDURE ScanAndMarkAttendance(qrToken, seminarId, adminUser):
  BEGIN TRANSACTION
  // 1. Lookup registration by cryptographic token
  reg = SELECT * FROM registrations
        WHERE qr_code_token = qrToken FOR UPDATE;

  IF reg IS NULL THEN
    ROLLBACK; RETURN REJECT("Invalid or Unrecognized QR Pass");
  IF reg.status ≠ 'confirmed' THEN
    ROLLBACK; RETURN REJECT("Pass is not confirmed");
  IF reg.seminar_id ≠ seminarId THEN
    ROLLBACK; RETURN REJECT("Pass belongs to another seminar");

  // 2. Check for duplicate scan collision
  att = SELECT * FROM attendance
        WHERE registration_id = reg.id;
  IF att EXISTS THEN
    ROLLBACK; RETURN REJECT("ALREADY CHECKED IN at " +
att.check_in_time);

  // 3. Record attendance & release transaction
  INSERT INTO attendance(registration_id, check_in_time, status,
marked_by)
  VALUES (reg.id, CURRENT_TIMESTAMP, 'present', adminUser.id);
  COMMIT;

  // 4. Dispatch notification & activate certificate eligibility
  TriggerCertificateGeneration(reg.id);
  SendConfirmationAlert(reg.user_id, "Attendance Verified");
  RETURN SUCCESS("Check-in Confirmed: Welcome " + reg.student_name);
END PROCEDURE
```

How it Works

The camera scan decodes the encrypted token string. A 4-layer validation pipeline checks token authenticity, active status, seminar scope, and duplicate guards in sub-100ms.

FIFO Waitlist Promotion

When a cancellation occurs, the engine locks the waitlist with `FOR UPDATE`, fetches the student at position 1, generates their pass, and notifies them via email automatically.

Inputs & Security State

Inputs: `qr_code_token`, `seminar_id`, `admin_id`. State: ACID compliant PostgreSQL relational commit.

2 6 Critical Security Checks & Conflict Resolvers

1. Duplicate Scan Guard

Prevents pass-sharing among students. Re-scans return previous timestamp.

2. Cross-Seminar Collision

Prevents a pass for Seminar A from being accepted at Seminar B's entrance.

3. Over-Capacity Race Condition

Row-level locking (`FOR UPDATE`) halts registrations once capacity hits 0.

4. Tamper-Proof Certificate Hash

Unique hash token prevents forging or photoshopping certificates.

5. Waitlist Queue Starvation

Ordered promotion guarantees first-come first-served queue fairness.

6. RBAC Privilege Escalation

JWT role middleware blocks unauthorized students from scanner endpoints.

3 Strengths

- Sub-second camera scan eliminates long paper entrance lines.
- ACID relational guarantees prevent overselling hall capacity.
- Fully automated certificate pipeline saves hours of admin work.
- Public verification portal provides instant employer trust.

4 Limits

- Live camera validation requires active network connectivity.
- Scanning speed dependent on venue lighting and camera focus.
- Email notifications require reliable external SMTP relay.
- Future: Offline scanner cache with delayed batch sync.

1 Test Cases & Validation Matrix

ID	Scenario	Expected Result
TC01	User Login	JWT Bearer token returned & stored
TC02	Create Seminar	Record saved; capacity quota set
TC03	Seat Booking	Registration created; capacity - 1
TC04	Over-Capacity	Student transferred to FIFO waitlist
TC05	Cancellation	Waitlist position 1 auto-promoted
TC06	QR Generation	Valid vector QR pass downloadable

ID	Scenario	Expected Result
TC07	Valid Camera Scan	Marked 'present'; audio chime
TC08	Duplicate Scan	Rejected: "Already checked in"
TC09	Wrong Seminar Scan	Rejected: "Pass belongs to other event"
TC10	Certificate PDF	Landscape PDF stream with verify QR
TC11	Public Verify Route	Valid certificate details returned
TC12	Database Health	/api/health confirms PG pool status

All 12 test cases verified against local and deployed Joy University environments.

2 Results

- 100% Paperless Passes
- Sub-100ms QR Verification
- Zero Duplicate Check-Ins
- Automated FIFO Waitlist
- Automated PDF Certificates
- Public Hash Verification
- Real-Time Turnout Roster

3 Limitations

- No offline PWA mode for zero-connectivity halls.
- Biometric / facial check-in is not yet integrated.
- Single campus scope; multi-tenant cluster planned.
- SMS alerts require commercial carrier gateway.

4 Future Enhancements

- [ ] Progressive Web App (PWA) with offline SQLite cache & sync
- [ ] AI recommendation engine matching seminars to student GPA & track
- [ ] Joy University ERP & RFID Smart Card contactless tap-in
- [ ] Live interactive Q&A and speaker rating polling during talks

5 Conclusion

JoySeminar transforms university seminar administration from disorganized paper slips and chaotic attendance lines into a streamlined, cryptographically secure full-stack web ecosystem. Featuring digital QR passes, real-time camera scanning, automated FIFO waitlists, and tamper-proof PDF certification with public hash verification, it establishes an enterprise foundation for Joy University's academic operations. Built with React 19, TypeScript, Express, and PostgreSQL.

6 References & Links

1. IEEE Standard for Software Test Documentation, IEEE Std 829–2008.
2. React 19 Documentation & Server Components Architecture, `react.dev`.
3. PostgreSQL Global Development Group, PostgreSQL 16 Reference Manual, `postgresql.org`.
4. PDFKit Vector Document Generation Engine, `pdfkit.org`.
5. Joy University Online Seminar Registration System snapshot, Academic Session 2026–2027.

Live Campus Portal: http://localhost:5173 API Health: http://localhost:5000/api/health Verification: /verify-certificate/:id